

# SpendWise – System Architecture Documentation

AI-Powered Expense Analyzer & Budget Assistant

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Project</b>  | SpendWise – AI-Powered Expense Analyzer and Budget Assistant                                                      |
| <b>Group</b>    | Group 13   Batch 7A                                                                                               |
| <b>Students</b> | Gitanshi Singh, Samiksha Tiwari, Shruti Mehkarkar, Viya Sharma, Abhiudaya Pratap Singh, Devbrat Yadav, Priyam Rai |

## 1. Introduction

This document presents the system architecture of SpendWise, an AI-powered expense analyzer and budget assistant. The architecture translates the functional and non-functional requirements identified during requirement analysis into a structured technical design. It explains the major layers of the system, the flow of information between them, and the individual software components responsible for authentication, expense management, budgeting, reporting, notifications, prediction, and data persistence.

## 2. Architecture Objectives

- Separate presentation, API, business logic, prediction, and data-access responsibilities.
- Provide a clear path for user requests from the web interface to the backend and database.
- Allow the prediction engine to use historical expense data without tightly coupling machine-learning logic to the user interface.
- Protect financial information through authentication, authorization, validation, and controlled data access.
- Make the system easier to maintain, test, extend, and scale.

## 3. High-Level System Architecture

The high-level architecture provides a simplified view of SpendWise and focuses on the major system layers rather than individual classes or modules. The main flow is: User → UI → API/Controller → Application Services → Prediction Service → Database, followed by a response back to the user.

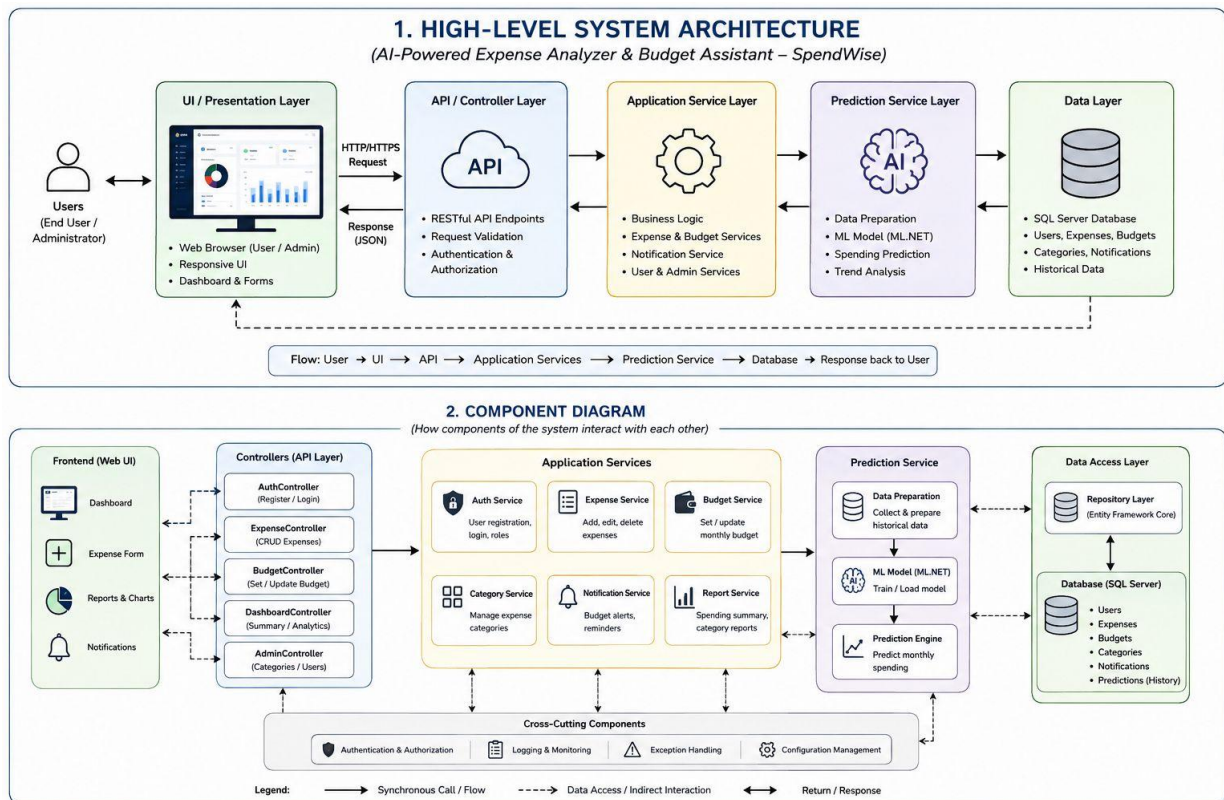


Figure 1. High-Level System Architecture of SpendWise

### 3.1 UI / Presentation Layer

The UI layer is the user-facing part of the application. End users can record expenses, set budgets, view dashboards, inspect reports and charts, and receive notifications. Administrators can access management functions such as user and category administration. The UI sends requests to the backend through HTTP/HTTPS and displays the returned results.

### 3.2 API / Controller Layer

The API/Controller layer acts as the entry point for backend operations. It exposes REST-style endpoints, validates incoming requests, performs authentication and authorization checks, and routes each request to the appropriate application service. The controller layer should not contain complex business logic; instead, it delegates work to the application services.

### 3.3 Application Service Layer

The application service layer contains the core business operations of SpendWise. It coordinates expense management, budget tracking, category management, notifications, reporting, and user administration. It also determines when the prediction service needs to be invoked and combines prediction results with budget information to support overspending warnings.

### 3.4 Prediction Service Layer

The prediction service is responsible for preparing historical expense data, applying the trained machine-learning model, and producing expected monthly spending estimates. The resulting prediction can be compared with the user's budget to identify the risk of exceeding it before actual spending crosses the limit.

### 3.5 Data Layer

The data layer provides persistent storage for users, expenses, budgets, categories, notifications, and prediction history. Historical expense data is particularly important because it provides the input used by the prediction service.

## 4. End-to-End Request Flow

| Step              | Description                                                                                       |
|-------------------|---------------------------------------------------------------------------------------------------|
| 1. User action    | The user adds an expense, updates a budget, opens the dashboard, or requests a report/prediction. |
| 2. UI request     | The web interface sends the request to the appropriate API endpoint.                              |
| 3. Controller     | The controller validates the request and forwards it to the corresponding application service.    |
| 4. Business logic | The application service performs the required operation and retrieves or updates data.            |
| 5. Prediction     | If required, historical data is prepared and passed to the ML prediction engine.                  |
| 6. Persistence    | Repositories retrieve or store the relevant records in SQL Server.                                |
| 7. Response       | The processed result is returned through the API and displayed by the UI.                         |

## 5. Component-Level Architecture

The component diagram expands the high-level architecture into concrete software components. It shows which controllers receive requests, which services perform business operations, how predictions are made, and how repositories communicate with the SQL Server database.

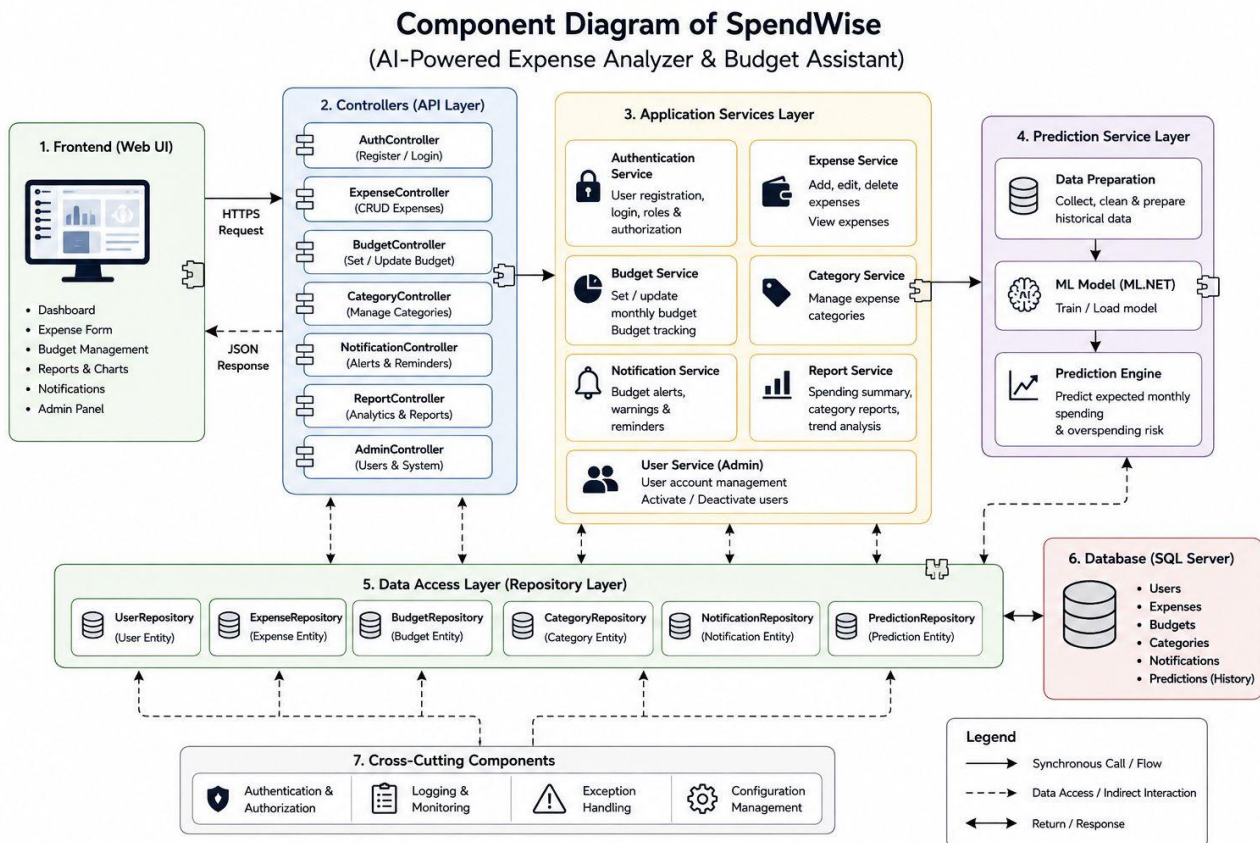


Figure 2. Component Diagram of SpendWise

## 5.1 Frontend (Web UI)

The frontend contains the dashboard, expense form, budget setup, reports and charts, notifications, and administrative views. It communicates with the backend through HTTPS and receives JSON responses.

## 5.2 Controllers / API Layer

The controller components provide focused entry points for different functions. AuthController handles registration and login; ExpenseController manages expense CRUD operations; BudgetController manages monthly budgets; CategoryController manages categories; NotificationController handles alerts and reminders; ReportController provides analytics and reports; and AdminController handles administrative user-management functions.

## 5.3 Application Services

Application services implement the business operations required by SpendWise. Authentication Service handles registration, login, roles, and authorization. Expense Service manages expense records. Budget Service manages budget setting and tracking. Category

Service manages categories. Notification Service creates budget alerts and reminders. Report Service produces spending summaries and category reports. User Service supports administrative account management.

## 5.4 Prediction Service

The Prediction Service contains three main parts: Data Preparation, the ML Model, and the Prediction Engine. Data Preparation collects and cleans relevant historical expense information. The ML Model is trained or loaded for prediction. The Prediction Engine uses the model to estimate expected monthly spending and identify potential overspending risk.

## 5.5 Data Access / Repository Layer

Repositories provide controlled access to persistent data. Separate repositories can be used for users, expenses, budgets, categories, notifications, and predictions. This layer keeps database operations separate from business logic and makes the system easier to test and maintain.

## 5.6 SQL Server Database

The database stores the system's persistent information, including users, expenses, budgets, categories, notifications, and prediction history. Historical records can be retrieved by the prediction service when generating future spending estimates.

## 5.7 Cross-Cutting Components

Authentication and authorization protect sensitive financial data and restrict access to authorized users. Logging and monitoring support troubleshooting and operational visibility. Exception handling manages unexpected failures, while configuration management keeps application settings centralized.

# 6. Component Interaction Summary

A typical expense workflow starts when a user submits an expense through the frontend. The request reaches ExpenseController, which delegates the operation to Expense Service. The service validates and processes the expense and uses ExpenseRepository to persist the record. The dashboard can then retrieve updated totals and category summaries. When a prediction is requested, the application service invokes the Prediction Service, which obtains historical data, prepares the data, applies the ML model, and returns the predicted monthly spending.

Notification Service can compare actual or predicted spending with the monthly budget and generate appropriate warnings.

# 7. Architecture Benefits

- Separation of concerns: each layer has a clear responsibility.
- Maintainability: individual services and controllers can be modified without changing the entire system.
- Security: authentication, authorization, validation, and controlled repositories protect financial information.
- Scalability: prediction and application services can be evolved independently as usage grows.

- Testability: business services and prediction logic can be tested separately from the UI and database.
- Extensibility: additional prediction models, reports, notification channels, or financial features can be added with limited impact on other components.

## 8. Conclusion

The SpendWise architecture converts the project's requirements into a clear layered design. The high-level architecture explains the overall system flow, while the component diagram identifies the internal modules and their interactions. Together, these diagrams provide the foundation for detailed database design, API development, machine-learning integration, testing, and Agile sprint implementation.